Faculty of Engineering and Technology

Department of Electrical and Computer Engineering

Artificial Intelligence ENCS3340

Project 1: Optimization Strategies for Local Package Delivery Operations

---

Prepared by: Lara Saleh Abu Asfour 1221484
Nadeen Jaber 1220044

Instructor: Yazan Abu Farha

Section: 4 & 1

Date: 4/5/2025

Introduction

This project focuses on optimizing package delivery for a small local shop that operates with limited delivery vehicles. Each package has a delivery location (defined by X and Y coordinates), a weight, and a priority level from 1 (highest) to 5 (lowest). Each vehicle has a maximum capacity, and the objective is to assign packages to vehicles in a way that minimizes the total travel distance while favoring the delivery of high-priority packages.

The problem is modeled as a variant of the Capacitated Vehicle Routing Problem with priorities, where delivering all packages is not mandatory lower-priority packages can be skipped if it results in a better overall cost. To solve this, we implemented and compared two heuristic algorithms: **Simulated Annealing (SA**) and **Genetic Algorithm (GA).** These algorithms iteratively search for improved delivery plans by exploring different package-to-vehicle assignments.

In addition to the optimization logic, we developed a user-friendly interface that allows users to upload delivery data, choose between the two algorithms, adjust settings, and visualize the resulting delivery routes. This system provides an effective and interactive tool for managing small-scale delivery logistics.

Project formulation

The problem consists of assigning each package to at most one vehicle and determining the optimal sequence of deliveries for each vehicle to minimize the total travel distance, while respecting the given constraints.

Solution Representation

- **Packages:** Each package p is represented by its ID, coordinates (px, py), weight pw, and priority pprio.
- **Vehicles:** Each vehicle v is represented by its ID and maximum capacity vcap in kilograms.
- **Solution:** A complete solution is represented as a list of Vehicle objects. Each Vehicle object contains a list of assigned Package objects, denoted as v.packages. The order of the packages within this list implicitly defines the sequence of delivery stops for that vehicle, starting from the shop (located at (0, 0)).

Objective Function

The goal is to minimize a cost function that balances the total distance traveled by all vehicles with the handling of package priorities. The objective function Cost(Solution) is defined as:

Penalty(Solution) + Total_Distance(Solution) = Cost(Solution)

Where:

- **Total_Distance(Solution):** The sum of the route distances for all vehicles in the solution. The distance for a single vehicle v is calculated using the Euclidean distance between consecutive stops, Distance(v) = Euclidean((0,0), (p1x, p1y)) + Σ Euclidean((pix, piy), (p(i+1)x, p(i+1)y)) + Euclidean((pnx, pny), (0,0)), Total_Distance = Σ Distance(v) for all vehicles v.
- **Penalty(Solution):** A penalty term added for packages that could not be assigned to any vehicle (due to capacity constraints or algorithmic choices). The penalty increases for higher-priority skipped packages, Penalty = Σ Penalty_Factor * (6 - pprio).

Constraints

- **Vehicle Capacity:** For each vehicle v in the solution, the sum of the weights of the assigned packages must not exceed its maximum capacity:

$$\sum_{p \in v.packages} pw \leq vcap$$

- **Package Assignment:** Each package should ideally be assigned to exactly one vehicle. Any unassigned packages will be penalized as described in the objective function.

Algorithms Used

Simulated Annealing (SA)

SA explores the solution space by iteratively transitioning from a current solution to a neighboring one. Moves that lead to better solutions (lower cost) are always accepted. However, moves to worse solutions may also be accepted with a probability that decreases as the temperature parameter decreases, helping the algorithm escape local optima during early iterations.

- **Initial Solution:** A random initial solution is generated by shuffling the package list and attempting to assign each package to a randomly selected vehicle that has sufficient remaining capacity. This ensures feasibility while encouraging diversity in the starting state.
- **Neighbor Generation:** A neighboring solution is created using one of two randomly selected operations: moving a package from one vehicle to another, or swapping packages between two vehicles.
- **Acceptance Rule:** Always accept better solutions. Accept worse ones with a probability that decreases as the temperature drops.
- **Cooling Schedule:** Starts at a temperature of 1000 and decreases every 100 iterations by multiplying it with a cooling rate (between 0.90 and 0.99). The process stops when temperature falls below 1.

Genetic Algorithm (GA)

GA maintains a population of candidate solutions and evolves them over generations using principles of natural selection and genetics.

- **Population Initialization:** A set of solutions is generated using a greedy-random method. Population size is user-defined (between 50 and 100).
- **Fitness Evaluation:** The fitness of each solution is calculated as the inverse of its total cost (distance + penalty).
- **Selection:** Tournament selection is used to choose parent solutions based on fitness.
- **Crossover:** A vehicle-based crossover operator assigns package lists from either parent to the corresponding vehicle in the child solution. Remaining unassigned packages are assigned greedily.
- **Mutation:** With a mutation rate (0.01–0.1), packages are randomly moved to different vehicles if capacity allows.
- **Elitism:** The best solution in each generation is preserved and passed to the next.
- **Termination:** The algorithm runs for a fixed number of generations (500).

Heuristics and Design Choices

Initial Solution Heuristic

Both SA and GA start with a randomized greedy assignment of packages to vehicles. This ensures capacity feasibility while maintaining solution diversity.

Route Ordering Simplification

The cost function assumes the order of package delivery follows the order in the vehicle's package list. Although applying a TSP solver per route could improve distance minimization, we chose not to include this due to its additional complexity.

Penalty-Based Priority Handling

We use a weighted penalty term in the objective function to guide the algorithms in handling package priorities. This enables flexible trade-offs between cost and service quality.

Constraint Handling

Vehicle Capacity (Hard Constraint)

Packages are only assigned to vehicles if the total weight stays within the vehicle's capacity. This is strictly enforced during initial assignment, crossover, and mutation. This check is performed using the method vehicle.can_add_package().

Package Assignment (Soft Constraint)

Delivering all packages is encouraged but not mandatory. Unassigned packages incur a penalty in the objective function—especially those with higher priority motivating the algorithm to prioritize important deliveries without sacrificing overall efficiency.

Parameter Tuning

Simulated Annealing: Cooling Rate

- **Effect:** A higher cooling rate (closer to 1.0) slows down the temperature decrease, allowing more exploration but increasing runtime. A lower rate (e.g., 0.90) speeds up convergence but may lead to suboptimal solutions.
- **Results:**
  - **α = 0.90:** Fast execution, but often produced higher-cost solutions.
  - **α = 0.99:** Sometimes found better solutions, but required much longer runtime.
  - **α = 0.95:** Offered the best balance between quality and speed.
- **Chosen Value: 0.95**, as it consistently delivered good results in a reasonable time.

Genetic Algorithm: Mutation Rate

- **Effect:** A low mutation rate preserves stability but may cause early convergence. A high rate maintains diversity but can slow down progress and disrupt good solutions.
- **Results:**
  - **0.01:** Fast convergence, but risk of poor diversity and weak solutions.
  - **0.1:** Maintained diversity but slowed convergence and caused fluctuations.

0.05: Balanced diversity and convergence effectively.

- **Chosen Value: 0.05**, as it provided reliable performance across different test cases.

Test cases

Complex test case

```
package.txt
# ID, X(km), Y(km), Weight(kg), Priority(1-5, 1=high)
1,90,90,10,1
2,75,15,18,1
3,5,10,5,1
4,10,15,8,3
5,20,8,7,4
6,50,50,12,2
7,30,75,6,3
8,65,60,9,4
9,40,30,20,5
10,95,5,8,4
11,5,95,7,4
12,80,35,10,3
13,60,80,11,2
14,25,45,9,5
15,55,15,15,3
```

Figure 1: Packages

```
vehicle.txt
# ID, Capacity(kg)
1,35
2,40
3,25
```

Figure 2: Vehicles

Optimization Results

```
Optimizer Ready. Load data to begin.
Loading packages from: package.txt
Loading vehicles from: vehicle.txt
Status: Loaded 15 packages, 3 vehicles.

==============================
Starting Optimization with SA
Parameters: {'algorithm': 'SA', 'initial_temp': 1000.0,
'cooling_rate': 0.95, 'stop_temp': 1.0, 'iter_per_temp': 100}
==============================

=== Optimization Results ===
Algorithm: SA
Best Cost Found (with penalty): 1628.59
  - Total Distance Cost: 628.59 km
  - Penalty Cost for Skipped: 1000.00

Vehicle Assignments:
  Vehicle 1: Cap=35.0, Load=33.0
    Packages: P7(W:6.0,Pri:3), P11(W:7.0,Pri:4), P13(W:11.0,Pri:2),
P8(W:9.0,Pri:4)
    Route Distance: 278.88 km
  Vehicle 2: Cap=40.0, Load=38.0
    Packages: P9(W:20.0,Pri:5), P10(W:8.0,Pri:4), P12(W:10.0,Pri:3)
    Route Distance: 231.28 km
  Vehicle 3: Cap=25.0, Load=20.0
    Packages: P15(W:15.0,Pri:3), P3(W:5.0,Pri:1)
    Route Distance: 118.44 km

Skipped Packages:
  - Package(ID=1, Loc=(90.0,90.0), Weight=10.0, Priority=1)
  - Package(ID=2, Loc=(75.0,15.0), Weight=18.0, Priority=1)
  - Package(ID=6, Loc=(50.0,50.0), Weight=12.0, Priority=2)
  - Package(ID=4, Loc=(10.0,15.0), Weight=8.0, Priority=3)
  - Package(ID=5, Loc=(20.0,8.0), Weight=7.0, Priority=4)
  - Package(ID=14, Loc=(25.0,45.0), Weight=9.0, Priority=5)
```

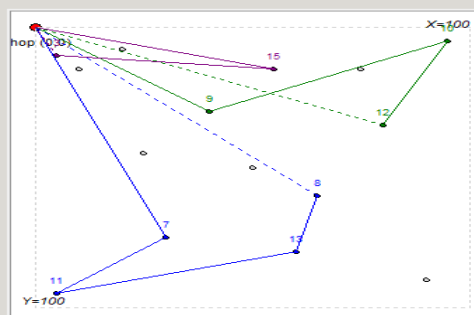


Figure 3: Simulated Annealing result

On the complex test case designed to stress the algorithms, Simulated Annealing found a solution costing 628.59. This cost was dominated by a 1000.00 penalty incurred by skipping six packages, including two critical Priority 1 packages (P1 and P2). This outcome illustrates SA's process under pressure, prioritizing feasibility and local cost reduction, which in this instance led to accepting significant penalties for omitting high-priority but difficult-to-route packages.

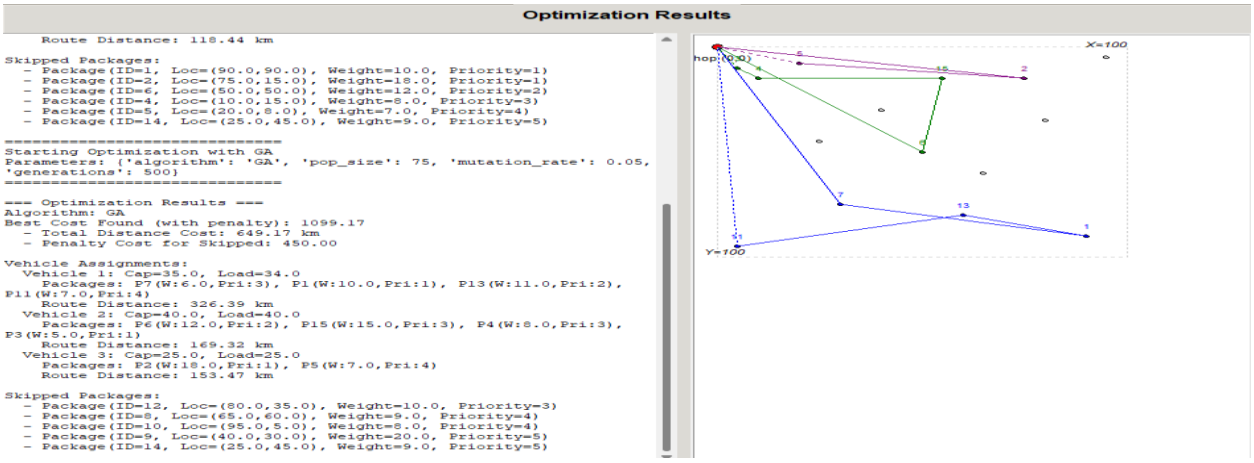
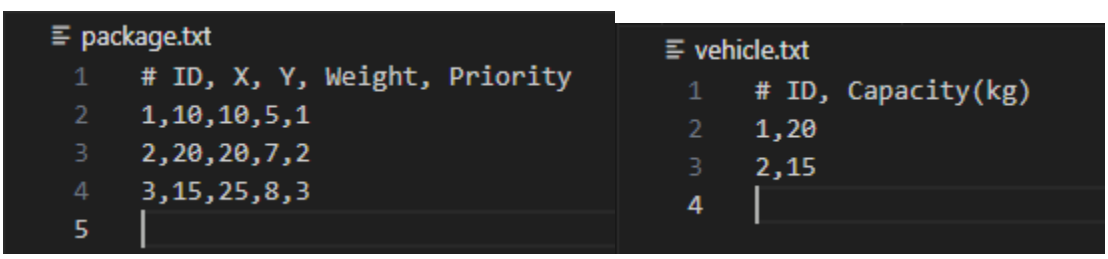
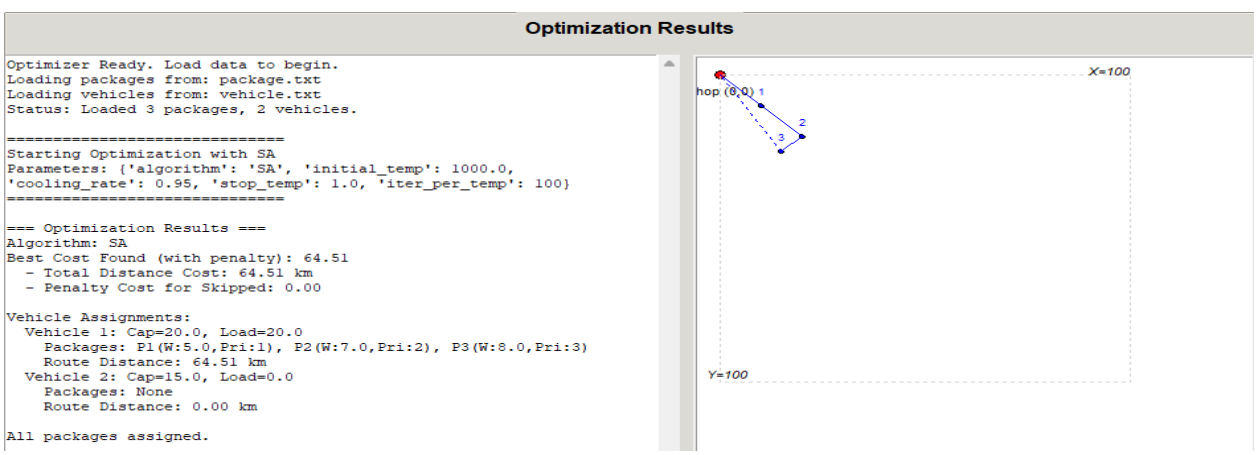


Figure 4: Genetic Algorithm result

On the complex test case, the Genetic Algorithm produced a solution costing 1099.17. By successfully delivering all Priority 1 packages, it significantly reduced the penalty cost to 450.00 compared to the SA run, despite a slightly higher travel distance of 649.17 km. The skipped packages were exclusively of lower priority (3, 4, and 5), showcasing GA's successful navigation of the distance-vs-penalty trade-off for this challenging scenario.

Simple test cases





In this test case all packages assigned to vehicles successfully, and penalty equals zero.

package.txt

```
# ID, X, Y, Weight, Priority
1,5,5,10,1
2,10,10,8,2
3,15,15,9,1
4,20,20,5,3
```

vehicle.txt

```
# ID, Capacity(kg)
1,10
2,10
```

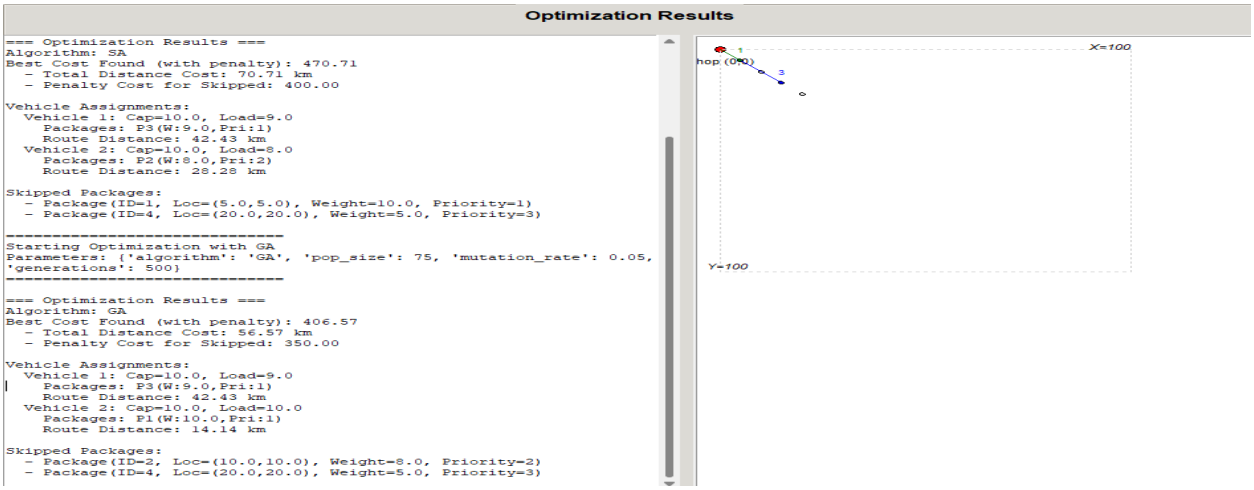


Figure 2: Simulated Annealing (SA) and Genetic Algorithm (GA) Result

In both algorithms, it is not possible to deliver all packages due to vehicle capacity constraints.

Conclusion

In tackling the MDVRP problem, our approach emphasized modularity, constraint management, and solution optimization through heuristics and met heuristics. By building a robust foundation with our problem formulation and initial heuristics, we ensured feasibility and scalability. Simulated Annealing and Genetic Algorithm were employed to navigate the solution space effectively, each bringing distinct strengths and tunable parameters to refine performance. The use of penalties for constraint violations enabled dynamic learning and prioritization in solution generation. Our testing affirmed the effectiveness of the approach in diverse scenarios, validating the adaptability and reliability of the implemented system.